

Project Yukon
V 1.0.0
Doc Version 0.0.1 (draft)

Commercial-in-confidence

About	3
Development Tools	3
Programming Criteria	3
Program Overview	4
Client Side (on cell phone or PDA).....	4
Admin Console (web browser)	4
Reporting.....	4
Automated Cron Jobs	4
Yukon Client Design notes	8
Yukon Clients web pages have a timer	9
Admin Console	9
Database.....	13
Application domain tables.	13
System domain tables	14
REPORTS	15
Alerts	15
Appendix A.....	16
Appendix B	19

About

Yukon is a subsystem of the PhantomEye “business spatial intelligence system”. The system allows a mobile worker to use a web browser to report his job status. The backend system allows an administrator to assign and schedule jobs.

Development Tools

PHP version 5.0.4 or use newer (if backward compatibility maintained)

MySQL version 4.1.20 – 5.xx

Programming Criteria

MySQL & PHP

- a) Database tables altering by way of adding new columns at end of existing columns should not crash any queries
- b) No framework like PEAR etc to be used, coding to be done using pure PHP constructs
- c) No use of session variables.
- d) PHP pages can call other php pages using curl
- e) Security to be achieved passing authenticated tokens
- f) All user interface should be XHTML compatible
- g) Use of CSS a must for the Yukon client.
- h) No debugging code to be built in.
- i) Access to MySQL as shown in attached script.(appendix a)
- j) Input and output checks for special characters like ‘ < > that may cause issues in XML rendering of data.

Program Overview

Client Side (on cell phone or PDA)

A user will have a browser enabled handheld device / cell phone through which he will access the Yukon System. The user front end for Yukon will look as shown in section Yukon Client. The client side will be simple xhtml /css web pages interface that must be adapted to run on the web browser of a cell phone. The design should allow for vertical scrolling and minimise any horizontal scrolling on the pages.

Admin Console (web browser)

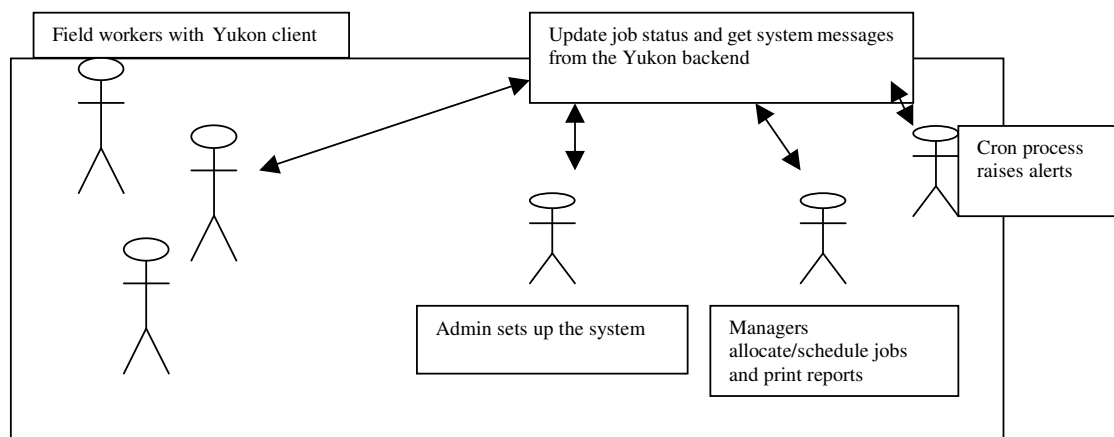
The admin console is a backend application where an administrator can enter job schedules, employee information and manage association between a device and a user.

Reporting

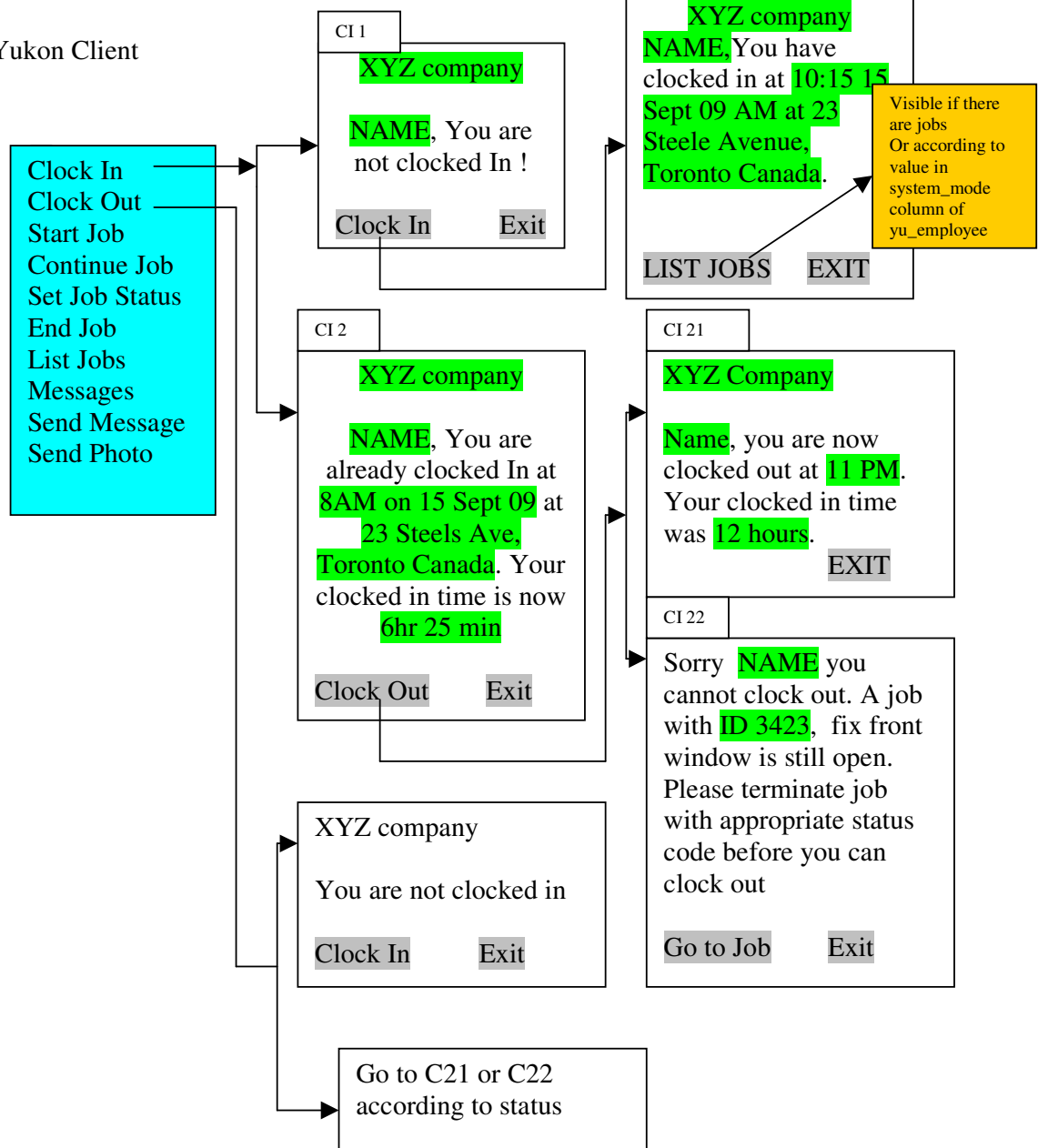
Reporting is also a function of backend application that will allow KPI reports. There is no reporting function on the client side on cell phone etc.

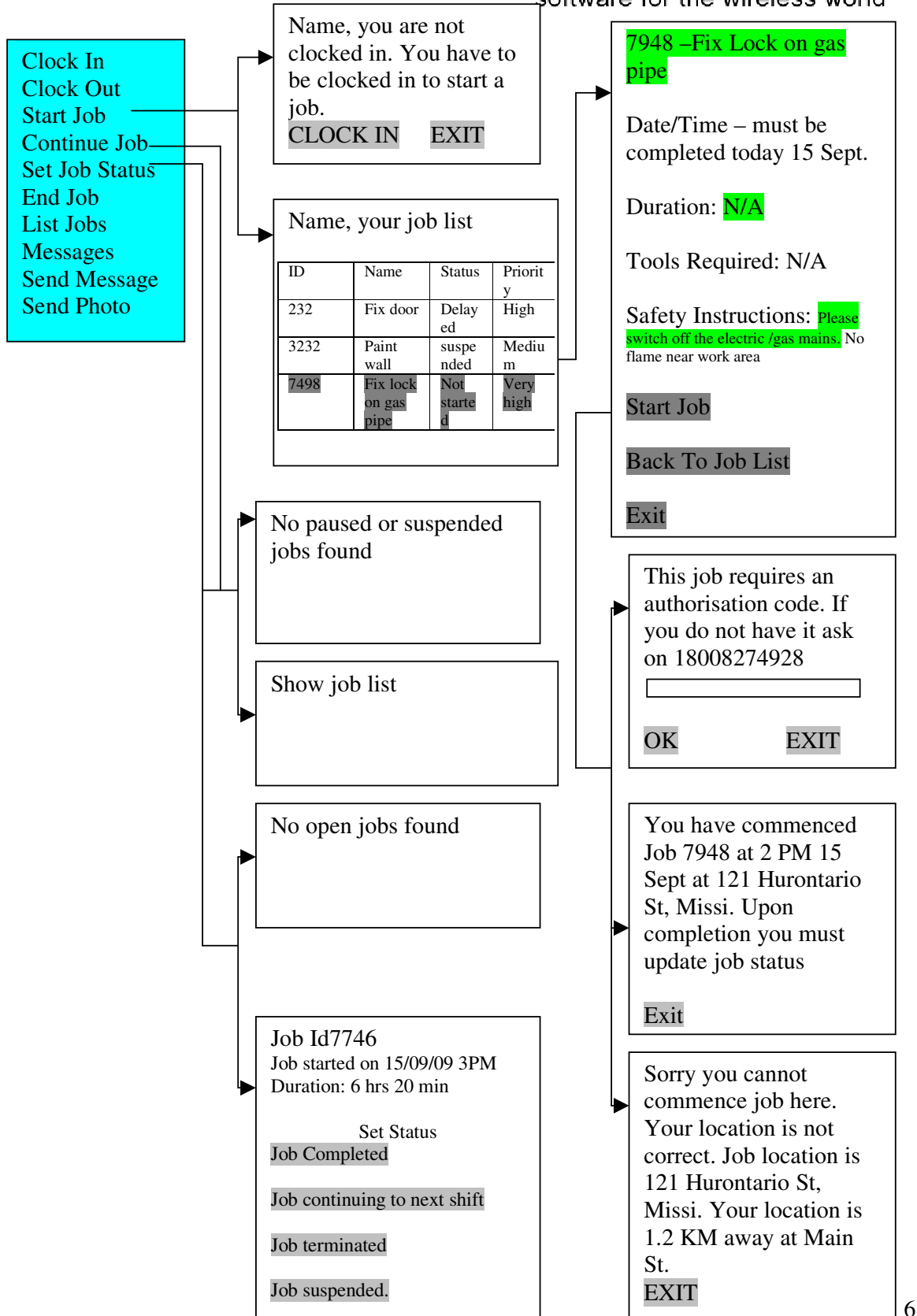
Automated Cron Jobs

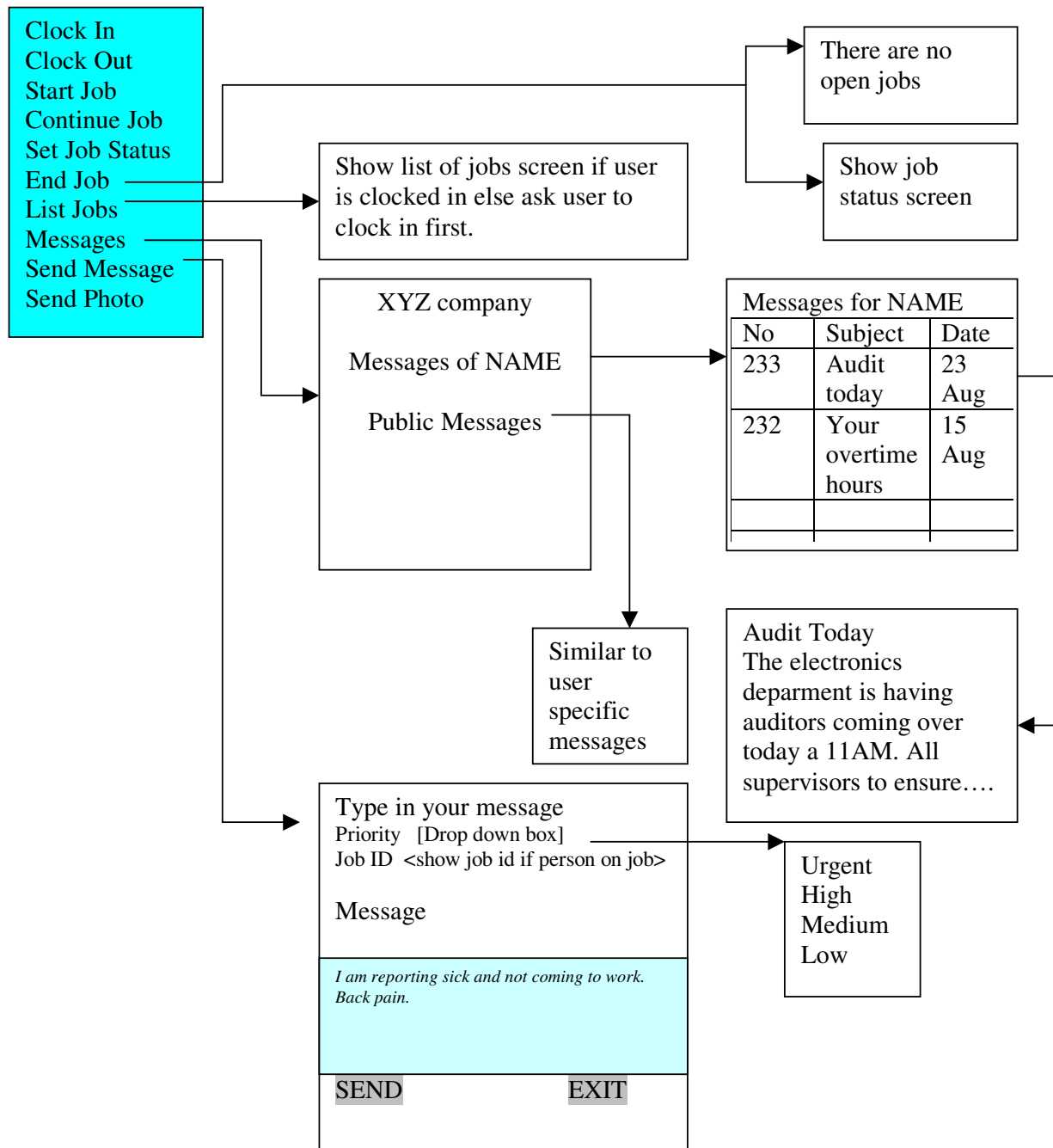
Certain alert functions of the system will require a backend cron to analyse the data in real time and find out late running jobs or exception conditions so that managers can be notified in time.



Yukon Client

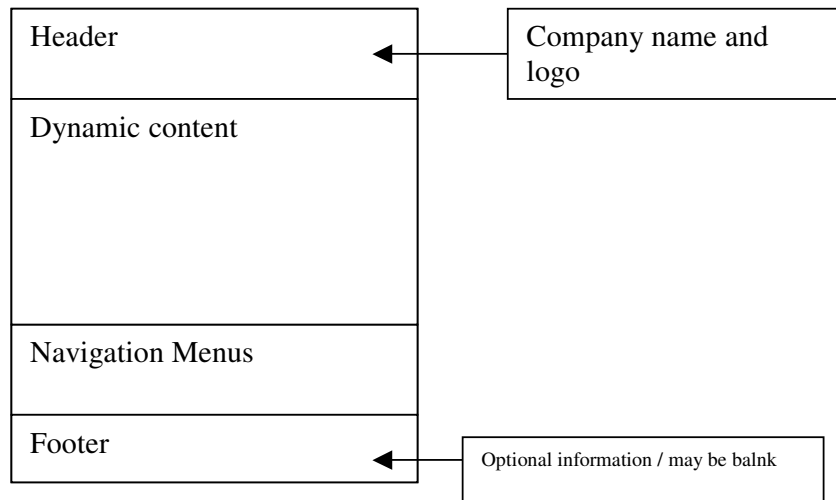






Yukon Client Design notes

The section above explains the screen navigation of the Yukon client. Each Yukon client screen is a web page that renders out xhtml/css. The design of the web page should be as follows



Clock In
Clock Out
Start Job
Continue Job
Set Job Status
End Job
List Jobs
Messages
Send Message
Send Photo

The menu in blue is another system (MFFM) that will make calls to the Yukon. This system is a native application on a device like blackberry or Nokia.

For the purpose of this development we will simulate calls from this system.

For example when the user click on “Clock In” on his device the menu will make a call to

<http://www.someserver.com/yukon/yukon.php?securityid=47834732&deviceid=82374287&action=clockin&lat=43.23232&lng=-55.34344&time=4832984392>

The structure of the php pages will be as follows

```
<?include database.php?>
<?include header.php?>
<?include footer.php?>
```

```
$token = $_GET['securityid'];
$time = $_GET['time'];
$lat = $_GET['lat'];
$lng = $_GET['lng'];
```

```
$security_check = checkAuthorisation($token,$time,$lat,$lng);
```

```
if($security_check != true) die (“You are not authorised”);
```

```
$getpagecreationtime = getUTCTimeInSeconds();
```

```
$action = $_GET['action'];
```

```
if($action == 'CLOCK_IN') {
```

```
// do the clock in stuff
```

```
//if you do a HTTP POST / GET to another php page here you must pass the $getpagecreationtime variable explained below.
```

```
}
```

for this development use the following security function

```
function checkAuthorisation($token,$time,$lat,$lng){
```

```
return true;
```

```
}
```

Yukon Clients web pages have a timer

Each time the client does a clock in or a clock out or any other action a web page is opened. The web page rendering time cannot be older than 5 minutes. Since we are not using a session variable to time out session we have to compare the page creation time stamp variable on each SUBMIT and check that \$getpagecreationtime is not older than 5 minutes.

The reason we want to limit to 5 minutes is so that the user may open the web page with the latest location co-ordinates. For example if the user opens the Clock In page at location X, but actually clocks in 20 minutes later at location Y then the location goes wrong. By giving a 5 minute time to each page we ensure that the location is always close by.

To do this follow the simple rule

- a) each time a page is created it should create a \$getpagecreationtime with a utc time stamp.
- b) Each time the submit is done from the page, the variable should be passed and checked by the destination php script. The destination php script should check if the passed creation time is not more the 5 minutes older to its creation time. If it is session timeout should be displayed and page exited.

Admin Console

The admin console will be simple html based interface (so that testing can be done – the final interface will be done in flash in phase 2 of the project) that will post data to php scripts and get data from php scripts. The following functions will be in the admin console.

All actions of admin will be logged. The Admin can create sub users and give them specific permissions.

I. Management Functions

- a) Admin login
- b) Create Sub users (managers who will use the system)
- c) Assign / Remove permissions to sub users.

II. System Function (only done by system admin)

- a) Set /edit company name
- b) Edit/create/delete departments
- c) Assign employees to departments (edit/create/delete)
- d) Assign phones to employees

III. Admin Functions (done by managers)

- a) Create/edit/delete a job type
- b) Create/edit/delete a job schedule and assign it to a user
- c) Send message to a user

The user interface is pure html to be used for testing only. All output has to be in XML format.

The following explains.

- a) Create a department

Create_department.html

```
<html>
<form action="create_department.php" method="post">
<!--put the userinterface element to add a departname and post>
```

```
</form>
</html>
```

We will just be using this interface for testing as the Admin interface will be built in the flash application and we will be calling the create_department.php and post variables from our flash program.

List_departments.php

```
echo "<units>";
while($rcount < $msg_rows )
{
    $id =mysql_result($main_result,$rcount,"id");
```

```
$name = mysql_result($main_result,$rcount,"name");  
  
echo "<unit><id>$id</id><name>$name</name></unit>";  
      $rcount++;  
  
      }  
  
    }  
    echo "</units>";
```

Please note that the format of output xml is like

```
<units>  
  <unit> <a> </a> <b> </b> <c> </c> </unit>  
  
</units>
```

The reason we have the units and unit is because the front end flash application is using data bound control that expect xml to have this structure.

For this development if you can produce the xml output for display of reports and user interface in the Admin section that is good enough.

Please note that this does not apply to the Yukon client.

The following interface is expected for the Admin Console of Yukon

A page with links to perform and check the following functions:

Admin

Log In For System Admin

Create a Manager Log In

Reset Password for Manager

Lock a manager login

Delete a manager login

List all managers logins name and date created.

Assign function permission to a manager login

The functions are

- a) Can create/edit/view job schedules
- b) Can only view/print job schedules
- c) Send messages

Add a new employee

Edit employee information

Delete employee

List all employee information

Add company information like name, phone, fax details

Edit company information
List company information.

Manager functions

Create a job
Edit/delete a job
Assign a job to a user
List all assigned jobs
List job history of an employee
Authorise a Job where authorisation is required
Change the status of a job

Database

Project Yukon is the start of project that will see many customer specific changes. It is very important that the MySQL data tables allow adding of new columns to tables without breaking any insert or update queries.

The following is the database structure to support the application

Application domain tables.

1. Company Table `yu_customer` (stores company information)

`Company_id, company_name, company_phone, late_at_work_limit_in_minutes, shift_length_in_hours` [the company id, will be passed from another program, for development purposes, use a dummy id]

2. Department Table `yu_department`

`company_id, department_id, department_name`

3. Employee Table `yu_employee`

`company_id, department_id, employee_id, employee_firstname, employee_middlename, employee_surname, manager_id, device_id, device_code, system_mode`

Note: The Yukon system has three modes

- a) Clock in /out only `YUKON_SIMPLE`
- b) Clock in/out with start/end job `YUKON_COMPLETE`
- c) (a) or (b) with client specific customisation `YUKON_CUSTOM`

Some employees may only be required to clock in/ clock out and not report on any specific jobs done during the clocked in time.

The Yukon client application is to show the interface to the user based on the setting in the system mode ie `SIMPLE / COMPLETE /CUSTOM`. Use an integer field to store these values.

4. Job Table `yu_jobs` (all new job orders are created and stored here)

company_id, department_id, job_id, job_title, job_order_no, client_id, job_location_lat, job_location_lng, authorisation_required, job_order_no, client_id, job_location_lat, job_location_lng, start_date, start_time, job_duration.

5. Assigned Jobs Table yu_assigned_jobs

company_id, department_id, job_id, employee_id

6. Job Status table yu_job_status

company_id, department_id, employee_id, job_id, job_status, update_time, manager_notes.

7. Attendance control yu_emp_attendance

company_id, department_id, clock_in_time, clock_in_lat, clock_out_lat, clock_out_time, clock_out_lat, clock_out_lng

System domain tables

1. Manager Login yus_manager

manager_id, password, locked

Please note that the database table design is NOT complete as all use cases have not been taken into consideration and just there to help explain what data model is required. You may be required to add additional tables or alter these tables. PLEASE DISCUSS BEFORE ANY DATABASE MODIFICATIONS.

Some columns like modified_time and modified_by have not been added and are required for auditing functions.

REPORTS

Yukon must be able to generate the following reports between two given dates
The output must be in xml format only as these reports will be piped into another program. The xml structure will be as follows
<units> <unit> xml tags </unit> </units>

- a) Job Assignments and Status ordered by date /client / employee /manager

Job name, start_time, end_time, no_of_hrs, status, employee, manager,client

- b) All jobs that are running late or finished late

Job name, start time,end time, delta, employee, client,location,reason

- c) Schedule report by day/week – print out next weeks scheduling

If by week

Dow, job name, employee, start_time, location,client

If by day

Start_time, job name, employee, location, client

- d) Clock in / out report for a day/week or for employee

For a day/week

Dow, date, employee, clock in time, clock out time, time at work

For employee

Date, clock in time, clock out time, time at work

THIS IS A WORK IN PROGRESS SECTION

Alerts

Alerts must be raised when the following happens

- a) Employee tries to start job from wrong location
- b) Employee start a job 10 min late
- c) Employee does not clock in
- d) Employee does not clock out after 8 hours.

For alerts a cron will run every 5 minutes and check for these 4 conditions. At this stage further thought is required on how it will work and this can be done at the end of the project.

Appendix A

Example PHP code

The coding has to vanilla, simple php constructs, direct MySql calls. Object oriented programming and use of objects / classes / interfaces OK – as long there is not too much of abstraction. Yukon is just starting, we have no idea where it goes, therefore it is best everything is kept simple and readable and EASY TO MODIFY.

```
<?
include("database.php");
```

```
$_userid = $_POST['userid'];
$_code = $_POST['code'];
```

```
$did=$_POST['deviceid'];
$devtype=$_POST['type'];
$tz=$_POST['tz'];
```

```
$date1=$_POST['date1'];
$date2 = $_POST['date2'];
```

```
$date1 = changedateformat($date1,'L');
$date2 = changedateformat($date2,'H');
```

```
$query = "SELECT DISTINCT date_format(starttime,'%d %b %Y') as
usedate,(endreading - startreading) as mileage
FROM `pe_day_startendsummary`
WHERE devid = $did
and starttime > '$date1' and starttime < '$date2' group by starttime ORDER BY
starttime desc";
```

```

$main_result = mysql_query($query, $connection);
if(!$main_result)
{

    die('error');
}
if($main_result)
{
    $msg_rows = mysql_num_rows($main_result);
    if($msg_rows == 0) {
        echo " ";
        die('No data ');}

    $rcount=0;

    echo "<units>";
    while($rcount < $msg_rows )
    {
        $date
=mysql_result($main_result,$rcount,"usedate");
        $mileage =
mysql_result($main_result,$rcount,"mileage");

        if($mileage < 0 ) $mileage = $mileage *
-1;

        if($mileage > 900000 ) $mileage = 0;

        $mileage = round($mileage * 0.010 *
1.609344,2) ;

        echo
"<unit><sno>$rcount</sno><date>$date</date><distance>$mileage</distance></uni
t>";

        $rcount++;

    }

}
echo "</units>";

?>

```

<?

```
function changedateformat($date1,$t)
{
$_year= substr($date1,6,4);
$_month= substr($date1,3,2);
$_day= substr($date1,0,2);
if($t=='L') { $_hour=0; $_minute=0; $_second=0;}
if($t=='H') { $_hour=23; $_minute=59; $_second=59;}
$date1 = date('Y-m-d H:m:s', mktime($_hour, $_minute, $_second, $_month, $_day,
$_year));
return $date1;
}
```

?>

Appendix B

Setting up development environment

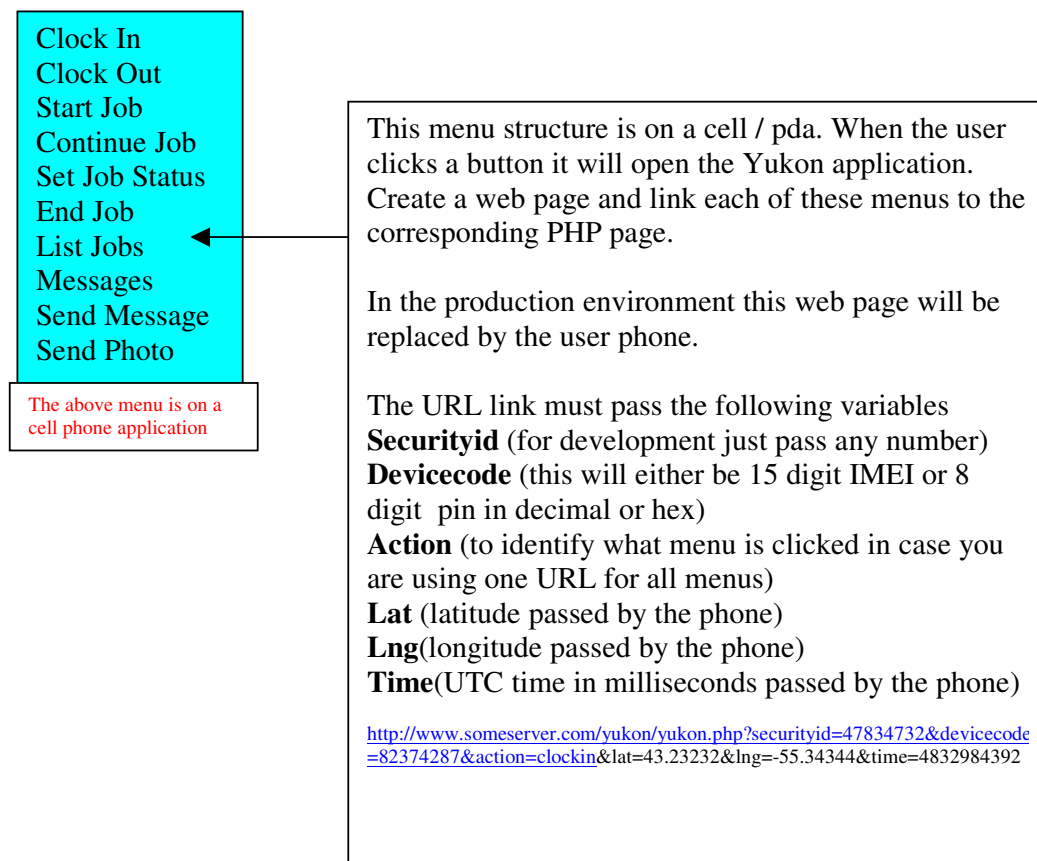
Backend folder structure

/Yukon (client web pages that open up on cell phone web browser)

/admin

/reports

For doing the Yukon client development, simulate a cell phone device by setting up a html page with links to the application you develop as suggested below



The device information is stored in the database of another system. For this development create the following table.

Customerid, device_name, device_imei, device_number, device_type,device_mode

129, puneets_bb, 43433434,14169312419,12,1

